

Merit-Ending

데이터분석/데이터 분석 및 시각화

[데이터 분석 및 시각화] 1. 시각화(Matplotlib 활용)

엔딩o 2025. 4. 4. 20:24

Matplotlib 파이썬에서 유명한 시각화 라이브러리
데이터 시각화 필요한 모든 기능 갖춘 도구

1. 라이브러리 불러오기

1) Matplotlib에서 한글을 쓰기위하여 한글표시 설치

```
# 한글 표시를 위해 설치  
!pip install koreanize_matplotlib -q
```

2) 실습에 필요한 라이브러리 불러오기

```
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import koreanize_matplotlib  
import warnings  
  
warnings.filterwarnings(action='ignore')  
%config InlineBackend.figure_format='retina'
```

2. 기본 차트 그리기

데이터를 읽어오고 전처리하는 과정은 생략하겠다!

```
plot()
```

1) y 축의 값만 지정하기

기본인덱스가 x가 되고 지정한 'Close'가 y축 값이 된다.

```
plt.plot(stock['Close'])
plt.show()
```

2) x축, y 축의 값 지정하기

stock 데이터의 'Day' 별 'Close'를 그래프로 그릴 수 있다.

```
plt.plot(stock['Day'], stock['Close'])
plt.show()
```

3. 그래프 꾸미기

1) x, y 축 이름과 타이틀 지정

xlabel(), ylabel(): x, y 축 이름 지정하기

title(): 그래프 제목 정하기

```
plt.plot(stock['Day'], stock['Close'])
plt.title('Stock Closing Price', size=15, pad=10)
plt.xlabel('Day')
plt.ylabel('Price')
plt.show()
```

2) 선 스타일 설정

선 스타일, 마커 형태

character	description
'-'	solid line style
'--'	dashed line style
'-.'	dash-dot line style
'.'	dotted line style
'.'	point marker
','	pixel marker
'o'	circle marker
'v'	triangle_down marker
'^'	triangle_up marker
'<'	triangle_left marker
'>'	triangle_right marker
'1'	tri_down marker
'2'	tri_up marker
'3'	tri_left marker

character	description
'4'	tri_right marker
's'	square marker
'p'	pentagon marker
'*'	star marker
'h'	hexagon1 marker
'H'	hexagon2 marker
'+'	plus marker
'x'	x marker
'D'	diamond marker
'd'	thin_diamond marker
' '	vline marker
'_'	hline marker

선 색상

character	color
'b'	blue
'g'	green
'r'	red
'c'	cyan
'm'	magenta
'y'	yellow
'k'	black
'w'	white

그래프 꾸미기 옵션

go--: 초록색 점선과 원 모양 마커이다.

순서 상관 없다 og--, --og, o--g 이런식으로 사용 가능

```
plt.plot(stock['Day'], stock['Close'], 'go--')
plt.title('Stock Closing Price', size=15, pad=10)
plt.xlabel('Day')
plt.ylabel('Price')
plt.show()
```

3) 여러 그래프 겹쳐서 그리기

show()로 마무리해주기 전에 plot()을 반복 출력하면된다.

```
plt.plot(stock['Day'], stock['Open'], 'go--')
plt.plot(stock['Day'], stock['Close'], 'rs--')
plt.title('Stock Price', size=15, pad=10)
plt.xlabel('Day')
plt.ylabel('Price')
plt.show()
```

동일한 코드

```
plt.plot(stock['Day'], stock['Open'], color='g', marker='o', linestyle='--')
plt.plot(stock['Day'], stock['Close'], color='r', marker='s', linestyle='--')
plt.title('Stock Price', size=15, pad=10)
plt.xlabel('Day')
plt.ylabel('Price')
plt.show()
```

4) 범례, 그리드(괘선) 추가

legend(): 범례를 표시해 이 데이터가 무엇을 나타내는 요소인지 이해 가능
label을 통해 지정

grid(): 데이터의 명확한 위치를 볼 수 있게됨

```
plt.plot(stock['Day'], stock['Open'], 'o--', label='Open')
plt.plot(stock['Day'], stock['Close'], 's--', label='Close')
plt.title('Stock Price', size=15, pad=10)
plt.xlabel('Day')
plt.ylabel('Price')
plt.legend() # loc='upper left' 등으로 위치 변경 가능, default='best'
```

```
plt.grid() # axis 지정 가능 ('both', 'x', 'y')
plt.show()
```

4. 추가기능

1) 축 범위 조정하기

xlim(), ylim(): 각 축에 표시할 범위 지정

y의 limit를 100 ~ 160로 지정하여 상대적으로 그 범위에 맞게 그래프를 그리게 한다.

```
plt.plot(stock['Day'], stock['Close'], marker='o')
plt.title('Stock Closing Price', size=15, pad=10)
plt.xlabel('Day')
plt.ylabel('Price')
plt.ylim(100, 160)
plt.show()
```

2) 그래프 크기 조정

figure() 함수를 통해 그래프 크기 조절

default: (6.4, 4.4)

너무 그래프가 크거나 작을 때 유용하다

```
plt.figure(figsize=(5, 5))
plt.plot(stock['Day'], stock['Close'], marker='o')
plt.title('Stock Closing Price', size=15, pad=10)
plt.show()
```

3) 수평선, 수직선 추가

axhline(), axvline() 함수를 사용해 임의 위치에 가로선과 세로선을 그린다.

특정 위치를 강조하고 싶을 때 (ex) 이 그래프의 평균은 ~이다!) 유용

```
plt.plot(stock['Day'], stock['Close'], marker='o')
plt.title('Stock Closing Price', size=15, pad=10)
plt.axhline(135, color='grey', linestyle='--')
plt.axvline(10, color='grey', linestyle='--')
plt.show()
```

4) 그래프에 텍스트 추가

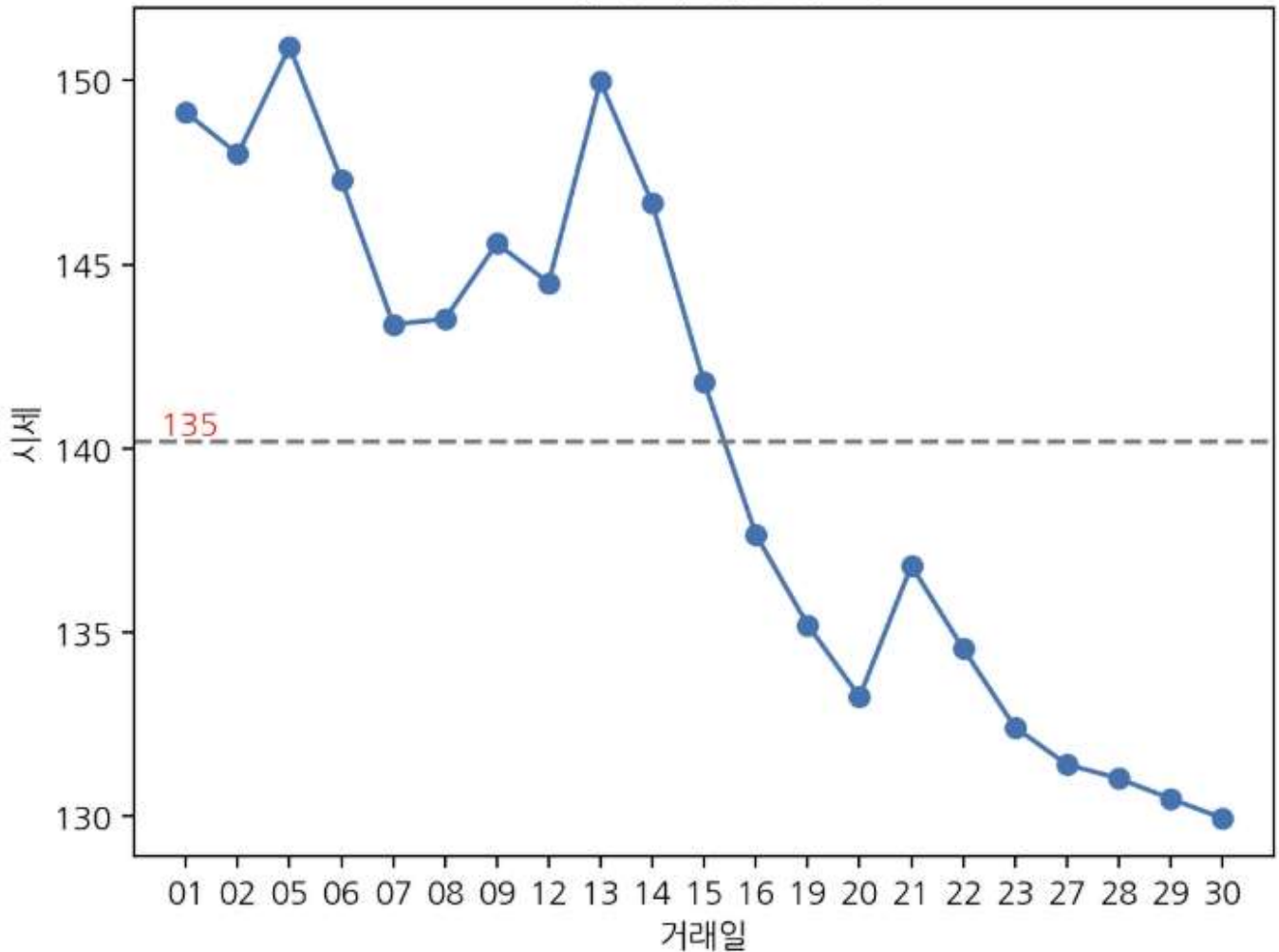
특정 위치를 강조하고 싶을 때 (ex) 이 그래프의 평균은 ~이다!) 유용

```
plt.plot(stock['Day'], stock['Close'], marker='o')
plt.title('Stock Closing Price', size=15, pad=10)
plt.axhline(135, color='grey', linestyle='--')
plt.axvline(10, color='grey', linestyle='--')
plt.text(-0.5, 135.2, '135', color='r')
plt.text(10.2, 125.5, '10', color='r')
plt.show()
```

※ 위의 예시:

```
y_mean= stock['High'].mean()
plt.plot(stock['Day'],stock['High'], marker='o')
plt.title('12월 주식 일별 최고가')
plt.xlabel('거래일')
plt.ylabel('시세')
plt.axhline(y_mean, color='gray', linestyle='--')
plt.text(-0.5, y_mean+ 0.2, 135, color='r')
plt.show()
```

12월 주식 일별 최고가



이런식으로 '주식 일별의 최고가의 평균은 135입니다.' 라고 강조 할 수 있다.
 중앙값, 표준편차 이런것도 실습해보기!

5. 여러 그래프 그리기

제일 유용한 개념!

subplot(): 하나의 칸에 여러 그래프를 그릴 수 있는 기능

subplot(행 수 , 열 수, 위치): 행렬 개념 이용하니까 명확하게 숙지해야된다.

1) 위, 아래로 그래프 그리기

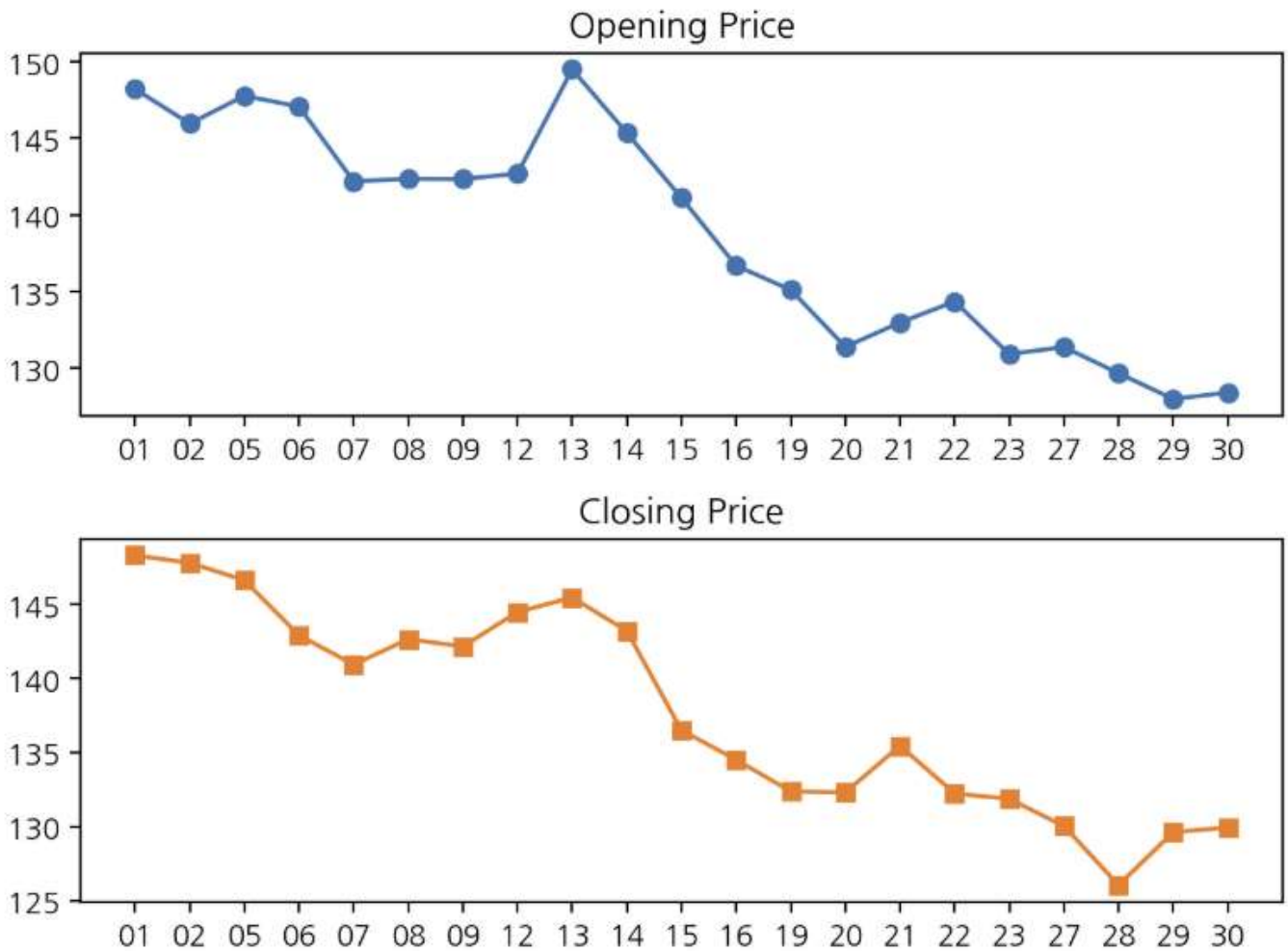
2x1 이니까 2행 1열로 그려지기에 위 아래로 그릴 수 있다!

```
plt.subplot(2, 1, 1)
plt.plot(stock['Day'], stock['Open'], color='tab:blue', marker='o')
plt.title('Opening Price')
```

```
plt.subplot(2, 1, 2)
plt.plot(stock['Day'], stock['Close'], color='tab:orange', marker='s')
```

```
plt.title('Closing Price')
```

```
plt.tight_layout()
plt.show()
```



2) 옆으로 그래프 그리기

반대로 1x2인 1행 2열로 그리기에 옆으로 그래프 나열하기 가능!

```
plt.subplot(1, 2, 1)
plt.plot(stock['Day'].astype(int), stock['Open'], color='tab:blue', marker='o')
plt.title('Opening Price')
```

```
plt.subplot(1, 2, 2)
plt.plot(stock['Day'].astype(int), stock['Close'], color='tab:orange', marker='s')
plt.title('Closing Price')
```

```
plt.tight_layout()
plt.show()
```

※ plt.tight_layout() 붙어야 그래프가 layout이 명확하게 분리되어 보여진다.

공감

구독하기

[데이터분석](#) > [데이터 분석 및 시각화](#) 카테고리의 다른 글

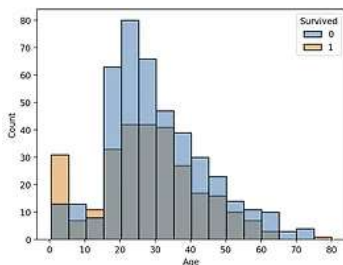
[\[데이터 분석 및 시각화\] 2. 시각화\(Seaborn 활용\)](#) (0)

2025.04.04

[\[데이터 분석 및 시각화\] 3. 단변량 분석 - 수치형](#) (0)

2025.04.01

'데이터분석/데이터 분석 및 시각화' Related Articles



NO IMAGE

[\[데이터 분석 및 시각화\] 2. 시각화\(Seaborn 활용\)](#) [\[데이터 분석 및 시각화\] 3. 단변량 분석 - 수치형](#)

Merit-Ending

엔딩o 님의 블로그입니다.

구독하기

댓글 0



이름

비밀번호

내용을 입력하세요.

등록